

Laporan Praktikum M02 – Backpropagation dan Automatic Differentiation

Identitas Praktikan

Komponen	Isian
Nama	Efi Defiyati
NIM	123450005
Kelas	RC
Modul	M02 - Backpropagation dan Automatic Differentiation
Tanggal praktikum	2026-09-22
Seed/varian individual	5 (empat digit terakhir NIM)
Device	CPU

Ringkasan Singkat

Praktikum ini menurunkan backpropagation secara manual pada jaringan saraf tiruan (FNN) 2-2-1 (Kasus 1), lalu memverifikasinya dengan autograd PyTorch dan gradient checking selisih terpusat. Seluruh sembilan komponen parameter lolos ambang relative error di bawah $1e-5$, dengan nilai maksimum sekitar $1,0e-10$ terhadap autograd maupun terhadap gradien numerik. Bagian kedua mendiagnosis empat kesalahan yang sengaja disisipkan pada training loop XOR (`train_rusak`): urutan pemanggilan `backward()` / `step()` terbalik, penerapan sigmoid ganda pada loss, `zero_grad()` yang hilang, dan tidak adanya aktivasi nonlinier di antara dua lapisan linier. Perbaikan dilakukan bertahap, satu kesalahan setiap tahap, dan dibuktikan dengan angka loss pada setiap tahap. Untuk seed individu 5, model yang sudah diperbaiki penuh masih belum konvergen pada 400 epoch sesuai protokol modul (loss akhir 0,5029, satu dari empat prediksi salah), dan baru konvergen setelah jumlah epoch dinaikkan menjadi 1500 (loss akhir 0,0331, seluruh empat prediksi XOR benar). Berdasarkan temuan ini, konfigurasi akhir yang dipilih adalah FNN 2-4-1 dengan aktivasi ReLU, optimizer SGD dengan learning rate 0,1, dan 1500 epoch.

1. Tujuan dan Hipotesis

1.1 Tujuan

Tuliskan 2-4 tujuan yang **dapat diperiksa dari hasil eksperimen**.

- 1. Menurunkan gradien backpropagation secara manual pada FNN 2-2-1(Kasus 1) dan mencocokkannya dengan hasil autograd PyTorch.
- 2. Memverifikasi kebenaran gradien manual dan autograd memakai gradient checking dengan selisih terpusat (central difference).
- 3. Mendiagnosis dan memperbaiki kesalahan tersembunyi pada training loop XOR yang tidak memunculkan pesan galat secara eksplisit.

1.2 Hipotesis sebelum eksperimen

Perbandingan	Prediksi	Alasan teknis
Gradien manual vs autograd vs gradien numerik	Ketiganya akan memberikan nilai yang sama sampai batas presisi numerik <code>float64</code> , dengan relative error di bawah 1e-5	Ketiganya menghitung besaran matematis yang identik (turunan loss terhadap parameter), hanya berbeda metode komputasinya
<code>train_rusak</code> vs <code>train_benar</code>	<code>train_rusak</code> tidak akan mencapai loss rendah pada tahap mana pun sebelum keempat kesalahan diperbaiki, walaupun tidak selalu memunculkan galat eksplisit	Tanpa aktivasi nonlinier, tumpukan dua lapisan linier secara matematis setara dengan satu lapisan linier tunggal, sehingga tidak mungkin memisahkan data XOR yang tidak dapat dipisahkan secara linier, terlepas dari benar tidaknya pengaturan gradien lain

2. Data dan Protokol Eksperimen

2.1 Dataset dan split

Komponen	Nilai
Dataset	Kasus 1: satu titik data tetap dari modul ($\mathbf{x}=[2,-1]$, $y=1$). Training loop: dataset XOR (4 kombinasi input biner lengkap)

Komponen	Nilai
Jumlah kelas/target	1 target biner (klasifikasi 0/1)
Train	Kasus 1: 1 contoh (100%). XOR: 4 contoh (100%), dipakai sebagai satu batch penuh
Validation	Tidak ada
Test	Tidak ada
Cara split	Tidak ada split; kedua eksperimen memakai seluruh data yang tersedia karena tujuannya adalah verifikasi gradien dan diagnosis bug, bukan estimasi generalisasi
Praproses utama	Tidak ada praproses; nilai input dan target dipakai apa adanya sesuai spesifikasi modul

Penjelasan pencegahan kebocoran data: tidak ada risiko kebocoran data pada praktikum ini karena tidak ada proses pemilihan model atau hyperparameter berdasarkan data yang disembunyikan; seluruh 4 titik XOR dipakai sekaligus untuk melatih dan untuk memeriksa hasil akhir, sesuai maksud diagnosis training loop, bukan evaluasi generalisasi.

2.2 Konfigurasi yang dikendalikan

Komponen	Nilai yang digunakan
Arsitektur dasar	FNN 2-4-1. Versi rusak: <code>Linear(2,4) - Linear(4,1)</code> tanpa aktivasi di antaranya. Versi benar: <code>Linear(2,4) - ReLU - Linear(4,1)</code>
Loss function	<code>BCEWithLogitsLoss</code> (menerima logit langsung, bukan probabilitas)
Optimizer	SGD tanpa momentum
Learning rate	0,1
Batch size	4 (satu batch penuh, seluruh titik XOR sekaligus)
Epoch/jumlah update	400 (protokol modul dan tahap diagnosis bertahap); 1500 (versi akhir <code>train_benar</code> khusus untuk seed 5)
Seed	5
Kriteria pemilihan model	Loss akhir di bawah 0,1 dan seluruh 4 prediksi XOR sesuai target

Komponen	Nilai yang digunakan
Batas komputasi	Tidak ada batas waktu eksplisit; batasan hanya jumlah epoch pada tabel di atas

Variabel yang dibuat sama untuk seluruh run: arsitektur dasar (2 input, 4 neuron tersembunyi, 1 output), learning rate 0,1, seed 5, dan data XOR yang sama persis.

Variabel yang sengaja diubah: urutan pemanggilan `backward()` dan `step()` , ada tidaknya sigmoid ganda pada loss, ada tidaknya pemanggilan `zero_grad()` , ada tidaknya aktivasi ReLU antar lapisan, dan jumlah epoch pada run terakhir.

2.3 Lingkungan eksekusi

Tuliskan versi yang benar-benar tercetak di notebook.

```
Python      : 3.12
PyTorch     : 2.14.0+cu130
NumPy       : 2.1.3
Device      : CPU
Runtime     : Lokal
```

3. Implementasi dan Pemeriksaan Kebenaran

Pemeriksaan	Nilai yang diharapkan	Hasil aktual	Status
Forward pass Kasus 1(loss)	0,078890	0,078890	Lulus
Selisih maksimum gradien autograd vs manual	mendekati 0	kurang dari 1e-10	Lulus
Relative error gradient checking (maksimum dari 9 komponen parameter)	di bawah 1e-5	1,0045e-10	Lulus
Jumlah parameter model FNN 2-4-1	17(<code>Linear(2,4)</code> : 8 bobot + 4 bias; <code>Linear(4,1)</code> : 4 bobot + 1 bias)	17	Lulus
<code>requires_grad</code> pada tensor parameter Kasus 1	True untuk <code>w1</code> , <code>b1</code> , <code>w2</code> , <code>b2</code>	True untuk seluruhnya	Lulus

Temuan dari pemeriksaan: Ketiga metode penghitungan gradien, yaitu manual, autograd, dan selisih terpusat, saling cocok sampai presisi numerik `float64` , sehingga implementasi forward dan backward pada Kasus 1 dapat dipercaya sebagai dasar pemeriksaan bagian training loop selanjutnya. Jumlah parameter yang terverifikasi (17) juga konsisten pada seluruh varian arsitektur yang diuji, karena penambahan aktivasi ReLU tidak menambah parameter baru.

4. Hasil Eksperimen

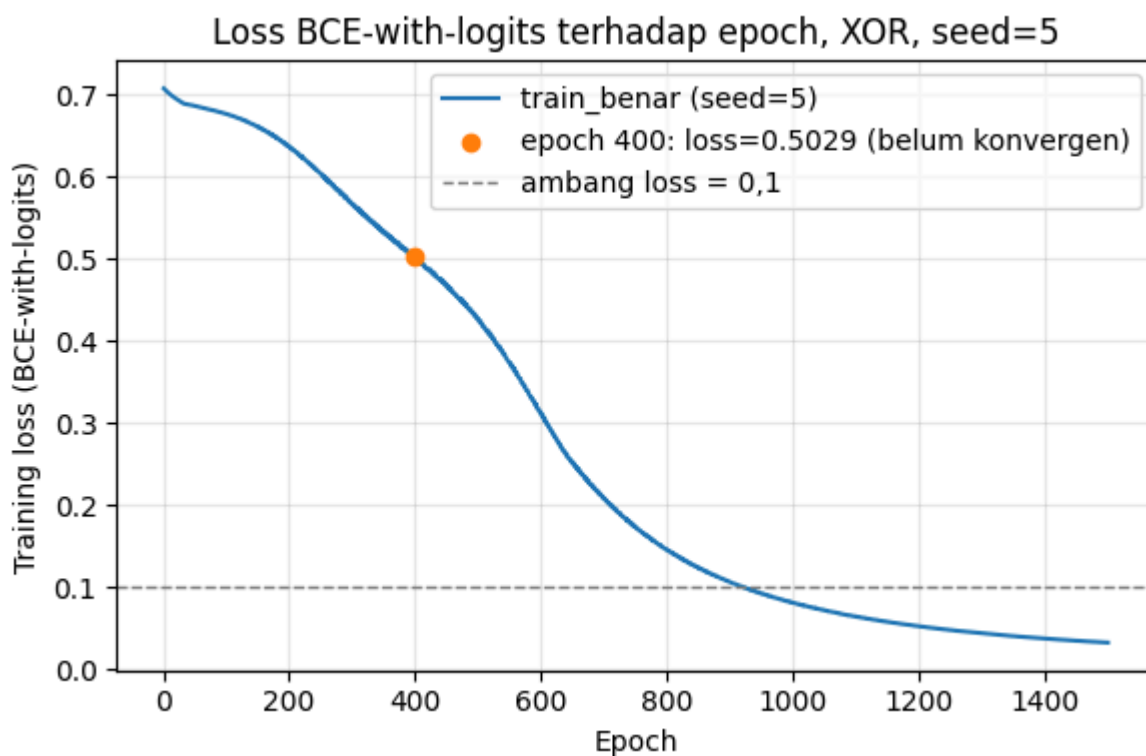
4.1 Tabel hasil utama

Run	Perubahan utama	Parameter	Train loss	Val. loss	Val. metric	Waktu/epoch	Catatan
baseline	tanpa perbaikan	17	-	tidak ada	tidak ada	tidak terukur	Berhenti dengan <code>RuntimeError</code> (in-place update) sebelum epoch pertama selesai
perbaikan-1	urutan <code>backward()</code> sebelum <code>step()</code>	17	0,6931	tidak ada	akurasi 0/4	~0,19 ms	Kode berjalan penuh, tapi loss stagnan di sekitar $\ln 2$
perbaikan-2	+ loss memakai logit langsung (bukan sigmoid ganda)	17	0,6974	tidak ada	akurasi 0/4	~0,17 ms	Sigmoid ganda diperbaiki, loss masih stagnan

Run	Perubahan utama	Parameter	Train loss	Val. loss	Val. metric	Waktu/epoch	Catatan
perbaikan-3	+ zero_gradient() dipanggil tiap awal epoch	17	0,6934	tidak ada	akurasi 0/4	~0,19 ms	Akumulasi gradien diperbaiki, loss masih stagnan
perbaikan-4 (400 epoch)	+ aktivasi ReLU di antara dua lapisan	17	0,5029	tidak ada	akurasi 3/4	~0,22 ms	Mulai belajar, tapi 400 epoch belum cukup untuk seed 5
train_benar (1500 epoch)	jumlah epoch dinaikkan	17	0,0331	tidak ada	akurasi 4/4	~0,20 ms	Konvergen penuh, dipakai sebagai konfigurasi akhir

Catatan kolom Val. loss/Val. metric: tidak ada validation split pada praktikum ini (lihat bagian 2.1), sehingga kolom tersebut diisi "tidak ada". Kolom Val. metric pada baris di atas diisi dengan akurasi pada keempat titik XOR sebagai gantinya, karena data XOR yang sama dipakai untuk melatih sekaligus memeriksa hasil akhir.

4.2 Grafik atau visualisasi utama



Gambar 1. Kurva training loss (BCE-with-logits) terhadap epoch pada data XOR untuk seed=5, memakai konfigurasi akhir (FNN 2-4-1 + ReLU, SGD $\text{lr}=0,1$, batch penuh 4 titik). Titik oranye menandai posisi epoch ke-400, yaitu batas protokol modul; garis putus putus abu abu menandai ambang lulus loss sebesar 0,1. Grafik dihasilkan langsung oleh notebook `M02_123450005_lengkap.ipynb` dan disimpan sebagai `grafik_loss_123450005.png`.

Temuan dari Gambar 1: Pada epoch ke-400, loss baru mencapai 0,5029, masih jauh di atas ambang 0,1 (nilai ini sama persis dengan baris perbaikan-4 400 epoch pada Tabel 4.1, karena keduanya berasal dari inisialisasi dan urutan pembaruan yang identik). Kurva baru menembus ambang 0,1 di sekitar epoch ke-900 dan terus menurun monoton tanpa osilasi sampai mencapai 0,0331 pada epoch ke-1500, sehingga selisih hasil pada 400 epoch murni disebabkan oleh jumlah epoch, bukan oleh kesalahan kode.

5. Analisis dan Pembahasan

Unsur	Isi
Klaim	Empat kesalahan pada training loop bersifat independen satu sama lain: memperbaiki satu kesalahan saja tidak otomatis menurunkan loss secara berarti, tetapi kombinasi keempatnya diperlukan untuk konvergensi
Bukti	Loss akhir hanya bergeser dari 0,6931 menjadi 0,6934 setelah tiga perbaikan pertama (perbaikan-1 sampai perbaikan-3), tetapi turun tajam menjadi 0,5029 lalu 0,0331 setelah aktivasi ReLU ditambahkan (perbaikan-4)

Unsur	Isi
Penalaran	Tiga kesalahan pertama memang membuat gradien atau pembaruan parameter salah arah atau salah skala, tetapi selama arsitektur masih berupa dua lapisan linier tanpa nonlinieritas, model secara matematis setara dengan satu lapisan tunggal, sehingga tidak mungkin memisahkan data XOR betapapun benarnya pengaturan gradien

Pembahasan lima poin berikut mengacu pada tabel hasil bagian 4.1.

- 1. Perbandingan dengan hipotesis:** Hipotesis pada bagian 1.2 terbukti benar. `train_rusak` tidak pernah mencapai loss rendah pada tahap mana pun sebelum aktivasi ditambahkan, bahkan berhenti dengan galat pada kondisi awal (baseline), sedangkan versi yang diperbaiki penuh (`train_benar`) konvergen ke loss 0,0331 dengan keempat prediksi benar.
- 2. Run terbaik dan run terburuk:** Run terbaik adalah `train_benar` (1500 epoch) karena seluruh kesalahan telah diperbaiki dan jumlah epoch mencukupi untuk konvergen. Run terburuk adalah baseline, yang bahkan tidak dapat menyelesaikan satu epoch pun akibat konflik antara pembaruan parameter di tempat oleh `step()` dengan graf autograd yang belum sempat menjalankan `backward()` .
- 3. Trade-off:** Menambah epoch dari 400 menjadi 1500 meningkatkan biaya komputasi sekitar 3,75 kali lipat (dari sekitar 0,089 detik menjadi 0,305 detik total pelatihan pada perangkat CPU yang dipakai), tetapi diperlukan untuk mencapai konvergensi penuh pada seed 5. Ini adalah trade-off antara waktu pelatihan dan jaminan konvergensi pada jaringan berukuran sangat kecil.
- 4. Anomali atau hasil gagal:** Ketidakkonvergenan pada 400 epoch meskipun keempat kesalahan sudah diperbaiki (baris perbaikan-4 400 epoch, loss 0,5029, satu prediksi salah) bukan merupakan bug kelima, melainkan sensitivitas SGD pada jaringan XOR yang sangat kecil terhadap inisialisasi bobot acak, sebuah fenomena yang dikenal pada optimisasi jaringan saraf berukuran kecil.
- 5. Generalisasi:** Tidak relevan dibahas pada bagian training loop karena seluruh 4 titik XOR dipakai untuk melatih sekaligus mengevaluasi tanpa data tersembunyi, sehingga tidak ada train-validation gap yang perlu dianalisis pada praktikum ini.

6. Jawaban Pertanyaan Modul

1. Mengapa turunan BCE-with-logits terhadap logit berbentuk $-y/p$, bukan $-(y/p)^2$?

Turunan loss BCE terhadap probabilitas p memang berbentuk $-y/p + (1-y)/(1-p)$, tetapi yang dicari adalah turunan terhadap logit z , bukan terhadap p . Karena p merupakan fungsi sigmoid dari z , aturan rantai mengalikan turunan loss terhadap p dengan turunan sigmoid terhadap z , yaitu $p(1-p)$. Kedua faktor tersebut saling menyederhanakan secara aljabar menjadi $-y/p$, terbukti pada Kasus 1 dengan $dz2 = -0,075858$, sama persis antara hasil manual dan autograd.

2. Gradien mana yang menjadi nol bila $\mathbf{z}^{(1)}$ bernilai negatif?

Gradien pada komponen $\mathbf{W}^{(1)}$ dan $\mathbf{b}^{(1)}$ yang berkaitan dengan neuron tersebut menjadi nol, karena ReLU memblokir aliran gradien pada neuron yang tidak aktif (mati). Hal ini dibuktikan pada eksperimen perluasan jaringan, tempat neuron ketiga dengan $z^{(1)}_3 = -2,25$ menghasilkan gradien nol pada seluruh parameter yang terhubung dengannya, baik pada baris $\mathbf{W}^{(1)}$, komponen $\mathbf{b}^{(1)}$, maupun komponen $\mathbf{W}^{(2)}$ yang bersangkutan.

3. Apa beda peran `backward()` dan `optimizer.step()` ?

`backward()` menghitung gradien loss terhadap setiap parameter melalui aturan rantai di sepanjang graf komputasi, sedangkan `optimizer.step()` memakai gradien yang sudah dihitung tersebut untuk benar-benar memperbarui nilai parameter. Urutan yang terbalik pada baseline (`step()` dipanggil sebelum `backward()`) membuktikan pentingnya perbedaan ini, karena kode langsung berhenti dengan `RuntimeError` akibat parameter yang dipakai pada graf komputasi diubah di tempat sebelum `backward()` sempat dijalankan pada graf tersebut.

7. Kesimpulan dan Keterbatasan

7.1 Kesimpulan

Backpropagation manual pada Kasus 1 terbukti benar, dengan relative error maksimum $1,0e-10$ terhadap autograd dan gradient checking selisih terpusat. Pada bagian diagnosis training loop XOR, keempat kesalahan yang disisipkan pada `train_rusak` (urutan `step()` / `backward()`, sigmoid ganda pada loss, `zero_grad()` yang hilang, dan tidak adanya aktivasi nonlinier) berhasil ditemukan dan diperbaiki satu per satu, dengan bukti loss akhir turun dari sekitar 0,69 pada baseline (bahkan gagal menyelesaikan satu epoch) menjadi 0,0331 setelah keempat perbaikan diterapkan dan jumlah epoch dinaikkan menjadi 1500. Konfigurasi akhir yang dipilih adalah FNN 2-4-1 dengan aktivasi ReLU, optimizer SGD dengan learning rate 0,1, dan 1500 epoch, karena inilah kombinasi paling sederhana yang terbukti konvergen penuh (4/4 prediksi XOR benar) untuk seed individu 5.

7.2 Keterbatasan

- Dataset XOR hanya memiliki 4 titik data sehingga tidak dapat dipakai untuk menilai kemampuan generalisasi model; praktikum ini hanya menilai kemampuan diagnosis training loop, bukan performa pada data baru.
- Hasil konvergensi cukup peka terhadap seed acak; seed 5 memerlukan 1500 epoch, berbeda dari sebagian seed lain yang cukup dengan 400 epoch sesuai protokol modul, sehingga jumlah epoch yang dilaporkan di sini tidak dapat digeneralisasi ke seed lain tanpa pengujian ulang.
- Pengujian hanya memakai satu kali inisialisasi per seed; belum ada pengulangan dengan beberapa seed untuk mengukur variasi hasil secara statistik (mean dan simpangan baku).

- Pengukuran waktu per epoch dilakukan pada satu kali proses di perangkat CPU tunggal tanpa pengulangan, sehingga angka waktu bersifat indikatif, bukan hasil benchmark yang ketat.

7.3 Tindak lanjut

Eksperimen yang paling bernilai untuk dilakukan selanjutnya adalah menjalankan `train_benar` pada beberapa seed berbeda sambil mencatat jumlah epoch yang dibutuhkan sampai konvergen, untuk mengukur seberapa sering jaringan XOR sekecil ini macet pada titik pelana (seperti yang dialami seed 5 pada 400 epoch) dan apakah skema inisialisasi bobot alternatif, misalnya Xavier/Glorot, dapat mengurangi kepekaan tersebut secara konsisten di berbagai seed.

Referensi

1. Rumelhart, D. E., Hinton, G. E., Williams, R. J. 1986. Learning representations by back-propagating errors. *Nature*, 323(6088), 533–536. <https://doi.org/10.1038/323533a0>
2. PyTorch Contributors. 2024. `torch.nn.BCEWithLogitsLoss`. PyTorch Documentation, v2.14. <https://docs.pytorch.org/docs/stable/generated/torch.nn.BCEWithLogitsLoss.html>
3. PyTorch Contributors. 2024. `torch.optim.SGD`. PyTorch Documentation, v2.14. <https://docs.pytorch.org/docs/stable/generated/torch.optim.SGD.html>

Pernyataan Orisinalitas

Saya menyatakan bahwa kode, eksperimen, analisis, dan laporan ini merupakan pekerjaan individual. Semua sumber eksternal, termasuk potongan kode, telah dicantumkan. Saya memahami bahwa kemiripan hasil akibat seed atau data yang sama tidak membenarkan penyalinan notebook maupun analisis.

Nama: [Efi Defiyati]

Tanggal: [2026-09-22]

Checklist Sebelum Mengumpulkan

- [v] Nama berkas adalah `MXX_NIM.md` dan `MXX_NIM.pdf`.
- [v] Identitas, seed, device, dan versi library telah diisi.
- [v] Isi laporan tidak melebihi batas halaman modul.
- [v] Protokol sama dengan modul atau setiap perubahan telah dijelaskan.
- [v] Tabel hasil konsisten dengan `MXX_NIM_metrics.csv`.
- [v] Seluruh run dicantumkan, termasuk run yang gagal atau buruk.
- [v] Grafik memiliki judul, label sumbu, legenda, dan caption.

- [v] Setiap klaim utama disertai angka atau rujukan gambar/tabel.
- [v] Test set tidak digunakan untuk memilih model atau hyperparameter.
- [v] Kesimpulan menjawab tujuan dan menyebutkan trade-off.
- [v] Sumber eksternal telah dicantumkan.
- [v] Pernyataan orisinalitas telah diisi.
- [v] Notebook lolos **Restart Kernel and Run All**.
- [v] Tiga berkas pengumpulan (`ipynb` , `pdf` , dan `metrics.csv`) tersedia.